A finite-difference Grad–Shafranov solver on a boundary-fitted non-homogeneous mesh: linear (Solov'ev) benchmark and nonlinear Picard equilibria

Emil H.V.

November 2, 2026

Abstract

We document a fixed-boundary Grad–Shafranov (GS) solver written in Python (NumPy/SciPy). The solver follows the finite-difference scheme on a boundary-fitted non-homogeneous mesh of López, Orozco and Dougar-Zhabon [1], which they validated only for the linear Solov'ev case, and extends it to the general *nonlinear* case with arbitrary $p'(\psi)$ and $FF'(\psi)$ through Picard iteration [2]. We describe the mesh construction, the discretisation, the sparse linear algebra and the iteration; we benchmark the linear solve against the analytic Solov'ev equilibrium (observed order of convergence ≈ 1.9, maximum error 7×10^{-6} on a 233×233 grid); we validate the nonlinear code path; we show nonlinear equilibria for several profiles and boundary shapes; and we study the spectrum of the Picard map, whose spectral radius (0.22–0.48 on the cases tested) predicts the observed 15–33 iterations well. We also record one practical lesson: the profile shapes must be imposed on p' and FF' so that the toroidal current density does not vanish on the magnetic axis; imposing them on p and F^2 and differentiating makes plain Picard oscillate on a period-two cycle.

Contents

1 Context **3**
- 1.1 What is the Grad–Shafranov equation? . . . 3
- 1.2 How should we interpret this equat ion? . . . 3
- 1.3 Fixed versus free boundary, direct versus inverse . . . 3
- 1.4 Scope of this work . . . 4

2 The solver **4**
- 2.1 Problem set-up and normalisation . . . 4
- 2.2 Inhomogeneous, boundary-fitted mesh . . . 5
- 2.3 Finite-difference discretisation . . . 6
- 2.4 Sparse linear algebra . . . 6
- 2.5 Picard iteration for the nonlinear case . . . 6

3 Results **7**
- 3.1 Benchmark against the Solov'ev solution . . . 7
- 3.2 Validation of the nonlinear code path . . . 8
- 3.3 Nonlinear equilibria: profiles and shapes . . . 9

4 Convergence of the Picard iteration **11**
- 4.1 Linear stability . . . 11
- 4.2 Measurements . . . 11
- 4.3 Profile parametrisation: why the shapes are on p' and FF' . . . 12
- 4.4 Outlook . . . 12

A Derivation of the Grad–Shafranov equation **12**
- A.1 Step I: vector potential and flux function . . . 13
- A.2 Step II: Ampère's law . . . 13
- A.3 Step III: force balance . . . 13
- A.4 Non-dimensionalisation . . . 14

1 Context

1.1 What is the Grad–Shafranov equation?

In a tokamak, the plasma is confined by a magnetic field that is, to a good approximation, symmetric under rotation about the machine axis. Writing (r, φ, z) for cylindrical coordinates and imposing $\partial_\varphi \equiv 0$ on all scalar fields, ideal MHD in *static* equilibrium reduces to

$$\boldsymbol{J} \times \boldsymbol{B} = \nabla p, \qquad \mu_0 \boldsymbol{J} = \nabla \times \boldsymbol{B}, \qquad \nabla \cdot \boldsymbol{B} = 0. \tag{1}$$

The divergence-free field is written through a poloidal flux function $\psi = rA_\varphi$ and a toroidal-field function $f = rB_\varphi$ as

$$\boldsymbol{B} = \frac{1}{r}\nabla\psi \times \hat{\boldsymbol{\varphi}} + \frac{f}{r}\,\hat{\boldsymbol{\varphi}}. \tag{2}$$

Force balance forces $p = p(\psi)$ and $f = f(\psi)$, and the remaining component of $\boldsymbol{J} \times \boldsymbol{B} = \nabla p$ is the Grad–Shafranov equation

$$\boxed{\Delta^*\psi = -\mu_0 r^2 \frac{\mathrm{d}p}{\mathrm{d}\psi} - f\frac{\mathrm{d}f}{\mathrm{d}\psi}}, \qquad \Delta^*\psi := \frac{\partial^2\psi}{\partial r^2} - \frac{1}{r}\frac{\partial\psi}{\partial r} + \frac{\partial^2\psi}{\partial z^2}. \tag{3}$$

It is a two-dimensional, elliptic, and (in general) *nonlinear* PDE for $\psi(r, z)$: it looks like a Poisson problem, but the source depends on the unknown through the two free functions $p(\psi)$ and $FF'(\psi) = \frac{1}{2}\mathrm{d}(f^2)/\mathrm{d}\psi$. A full derivation is given in Appendix A.

1.2 How should we interpret this equat ion?

- **Shape of the confinement region.** Level sets of ψ are the magnetic flux surfaces: nested toroidal surfaces on which field lines lie and, to leading order, along which heat and particles flow quickly. The equilibrium tells us where the plasma sits, its elongation and triangularity response, the Shafranov shift of the magnetic axis, and the location of the separatrix and X-points.
- **Starting point for everything else.** The safety factor $q(\psi)$, the shear, the stability of MHD modes (ballooning, kink, tearing), transport codes, particle-orbit studies and heating/current-drive calculations all take an equilibrium as input.
- **Operation and design.** Real-time reconstruction codes solve GS from magnetic measurements to control the plasma shape; design studies use it to explore shapes and profile scenarios before building anything.
- **A cheap but rich numerical problem.** It is nonlinear, elliptic and posed on a non-trivial domain, which makes it a good testbed for meshing, sparse solvers and nonlinear iteration.

1.3 Fixed versus free boundary, direct versus inverse

Fixed boundary. The last closed flux surface is *prescribed* (here by a parametric D-shape, or by a Solov'ev contour), $\psi = \psi_b$ is imposed on it, and (3) is solved only inside. The exterior is irrelevant. Choosing the profiles $p(\psi)$, $FF'(\psi)$ and the shape then gives ψ, and in a real device the pressure profile and toroidal field are controllable to a good extent, so scanning shape and profile weights is an inexpensive way to build intuition. This is the problem treated here.

Free boundary. The plasma is surrounded by external coils; $\Delta^*\psi$ is solved on a large computational box with the coil currents as sources, and the plasma boundary is actually *part of the solution* (the contour through the X-point or touching the limiter). At each iteration the boundary is recomputed, which captures the natural plasma displacement and gives a much more physical description, at the price of significantly more machinery.

Inverse (reconstruction) problems. Given measurements (e.g. magnetic probes on JET), one fits p' and FF' parametrically so that the GS solution matches the data; this is how $p(\psi)$ and $f(\psi)$ are deduced from experiment.

1.4 Scope of this work

The reference for the numerical scheme is López *et al.* [1]: a fixed-boundary, D-shaped, finite-difference solver on a non-homogeneous mesh, with SOR iteration, validated against the Solov'ev solution. Their validation is a *linear* problem (the right-hand side is a fixed function of position). Here we reproduce that benchmark and then go further: the code also handles the *nonlinear* source $S(r,\psi)$ for arbitrary $p'(\psi)$, $FF'(\psi)$, via a Picard iteration, and we analyse the behaviour of that iteration.

2 The solver

All code is in `src/gs_solver.py`; the optional diagnostics used in Section 4 are in `src/gs_fixedpoint.py`. The solver has three interchangeable modes: `SOLOVIEV_LINEAR` (one linear solve), `PICARD_NONLINEAR` (the general case) and `CUSTOM` (user-supplied source), combined with either of two boundary geometries.

2.1 Problem set-up and normalisation

Lengths are normalised to the major radius R_0. With the flux normalised to

$$\psi_n = \frac{\psi - \psi_{\text{axis}}}{\psi_b - \psi_{\text{axis}}} \in [0,1], \tag{4}$$

so that $\psi_n = 0$ on the magnetic axis and $\psi_n = 1$ on the boundary, the equation solved is

$$\Delta^*\psi = S(r,\psi_n), \qquad \psi = \psi_b \text{ on the boundary}, \qquad \partial_z\psi = 0 \text{ on } z = 0. \tag{5}$$

The second condition exploits up–down symmetry: only the upper half-plane $z \geq 0$ is discretised and the field is mirrored for plotting.

Boundary geometries.

- *Shafranov (parametric, Miller-type)*: $R(t) = R_0 + a\cos\big(t + \delta\sin t\big)$, $Z(t) = \kappa a \sin t$, with minor radius a, triangularity δ and elongation κ.
- *Solov'ev*: the closed D-shaped contour $Z^2(R) = \big[a_s - b(R^2 - R_0^2)^2\big]/R^2$ with $b = \kappa_0^2/4$, $a_s = 2R_0^2\kappa_0 q_0 \psi_b/B_0$ (parameters κ_0, q_0, B_0 as in [1]). It is the $\psi = \psi_b$ level set of the exact solution, so the boundary and the source are consistent.

Source terms. The profile shapes are imposed directly on the *derivatives* p' and FF', as functions of ψ_n,

$$-\frac{\mathrm{d}p}{\mathrm{d}\psi_n} = p_0\,(1-\psi_n^{s_{p1}})^{s_{p2}}, \qquad -\frac{\mathrm{d}(F^2)}{\mathrm{d}\psi_n} = \alpha\,(1-\psi_n^{s_{f1}})^{s_{f2}}, \tag{6}$$

both equal to 1 (times the amplitude) on the axis and to 0 on the boundary; the exponents control how peaked or broad the profiles are. With $FF' = \frac{1}{2}\,\mathrm{d}(F^2)/\mathrm{d}\psi$ the normalised source is

$$S(r,\psi_n) = -r^2\frac{\mathrm{d}p}{\mathrm{d}\psi_n} - \frac{1}{2}\frac{\mathrm{d}(F^2)}{\mathrm{d}\psi_n} = r^2 p_0\,(1-\psi_n^{s_{p1}})^{s_{p2}} + \frac{\alpha}{2}(1-\psi_n^{s_{f1}})^{s_{f2}} \;\geq 0. \tag{7}$$

The sign matters: $S \geq 0$ means $\Delta^*\psi \geq 0$, so ψ is smallest in the interior (the magnetic axis) and increases towards the boundary, consistent with $\psi_n = 0$ on the axis; with $S < 0$ the "minimum"

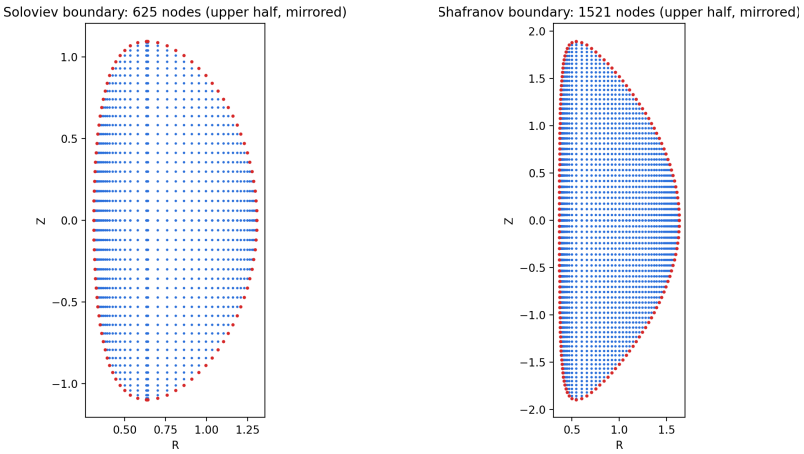


Figure 1: Non-homogeneous boundary-fitted meshes (upper half computed, mirrored for display). Red: Dirichlet nodes, blue: interior nodes. Left: Solov'ev contour; right: Shafranov parametric boundary ($\kappa = 3$, $\delta = 0.8$).

that the code identifies as the axis lands on the boundary. The toroidal current density is $\mu_0 r J_\varphi = -S$, so, since S does not vanish at $\psi_n = 0$, the current density is non-zero on the axis (a peaked-current scenario; hollow-current configurations are outside the scope of this work). Note that $\mu_0 J_\varphi \propto S/r = r p_0\, p_{\text{shape}} + \frac{\alpha}{2r} f^2_{\text{shape}}$: the $1/r$ and r factors move the maximum of J_φ off the axis towards the outboard or inboard side depending on p_0/α (Fig. 6).

Why the shapes are on the derivatives. López *et al.* write the profiles as $p = p_0 \tilde{p}(\psi_n)$ and similar for f^2, i.e. with shape functions on p and f^2 themselves. Imposing the shapes $(1-\psi_n^a)^b$ on p and differentiating makes p' vanish on the axis for $a \geq 2$, hence a vanishing source and a hollow current profile there, and in our experiments plain Picard then does not converge (Section 4.3). Shaping p' and FF' directly avoids the issue.

For the linear Solov'ev mode the source is the position-only function $S = (8b+2)\, r^2/a_s$, whose exact solution is $\psi(r,z) = \psi_b\, [\, b(r^2 - R_0^2)^2 + r^2 z^2\,]/a_s$ [1].

2.2 Inhomogeneous, boundary-fitted mesh

A uniform Cartesian grid represents a curved boundary badly (staircase errors of order h unless special near-boundary stencils are used). Following [1], the grid is instead built so that boundary points *are* grid nodes. Figure 1 shows the result for both geometries.

1. **Outer boundary at constant arc length.** Starting at the outboard midplane ($Z = 0$) the boundary is walked towards the top, placing points at (approximately) constant arc-length spacing Δs. Given the last point (R_k, Z_k) the next one solves $f(\tau) = \|\boldsymbol{x}(\tau) - \boldsymbol{x}_k\|^2 - \Delta s^2 = 0$ in the curve parameter τ (Newton with a Brent fallback), as in López's Eq. (18). The sampled heights z_k define the z-mesh.
2. **Inner boundary at the same heights.** For every z_k the radius of the inner (high-field-side) boundary is found by one-dimensional root finding.
3. **Tensor grid and mask.** The R-mesh is the union of all inner and outer boundary radii; the grid is the tensor product R-mesh $\times$ z-mesh, and a node is *active* if it lies between the inner and outer radius of its row. Row by row, the first and last active nodes therefore sit exactly on the boundary.

The mesh is non-uniform in both directions (finer where the boundary curves strongly), and Δs is the single resolution knob. The number of unknowns is $N \sim 10^2$–10^4 in the experiments below.

2.3 Finite-difference discretisation

On a non-uniform 3-point stencil with $h_+ = x_{i+1} - x_i$, $h_- = x_i - x_{i-1}$ and $h_s = h_+ + h_-$, the second-order accurate approximations are

$$f_i'' \approx \frac{2}{h_s}\left[\frac{f_{i+1} - f_i}{h_+} + \frac{f_{i-1} - f_i}{h_-}\right], \tag{8}$$

$$f_i' \approx \frac{h_-^2\,(f_{i+1} - f_i) - h_+^2\,(f_{i-1} - f_i)}{h_+ h_- h_s}. \tag{9}$$

(The leading truncation error of the first-derivative formula is $\frac{1}{6}h_+h_-f'''$, so it is second order in the local mesh size; the second-derivative formula is first order at nodes with $h_+ \neq h_-$ and second order on smoothly graded meshes, which is the case here.) Applying these in r for $\partial_r^2 - \frac{1}{r}\partial_r$ and in z for ∂_z^2 gives, at an interior node (i, j) with $r = r_i$,

$$\Delta^*\psi\big|_{ij} \approx c_{R+}\psi_{i+1,j} + c_{R-}\psi_{i-1,j} + c_{Z+}\psi_{i,j+1} + c_{Z-}\psi_{i,j-1} + c_0\psi_{ij}, \tag{10}$$

with, in terms of $h_\pm^{(r)}$, $h_s^{(r)}$ and $h_\pm^{(z)}$, $h_s^{(z)}$,

$$c_{R+} = \frac{2}{h_+^{(r)} h_s^{(r)}} - \frac{(h_-^{(r)})^2}{r\,h_+^{(r)} h_-^{(r)} h_s^{(r)}}, \qquad c_{R-} = \frac{2}{h_-^{(r)} h_s^{(r)}} + \frac{(h_+^{(r)})^2}{r\,h_+^{(r)} h_-^{(r)} h_s^{(r)}},$$

$$c_{Z\pm} = \frac{2}{h_\pm^{(z)} h_s^{(z)}}, \qquad c_0 = -\frac{2}{h_+^{(r)} h_-^{(r)}} + \frac{(h_-^{(r)})^2 - (h_+^{(r)})^2}{r\,h_+^{(r)} h_-^{(r)} h_s^{(r)}} - \frac{2}{h_+^{(z)} h_-^{(z)}}.$$

This is the same structure as López's Eq. (14), written for the operator in the form used in this code.

Boundary conditions.

- *Midplane* ($z = 0$, symmetry): a ghost value $\psi_{i,-1} = \psi_{i,1}$ gives $\partial_z^2\psi|_{z=0} \approx 2(\psi_{i,1} - \psi_{i,0})/(h_+^{(z)})^2$.
- *Boundary*: a node is *Dirichlet* ($\psi = \psi_b$, which is 1 in normalised units) if it is on the top row, or has a missing active neighbour in the r-direction (inner/outer wall), or a missing neighbour in z (except at the midplane, where the ghost value is used). Dirichlet rows of the matrix are identity rows. Because the grid lines pass through the boundary points in R, the radial condition is imposed exactly; a node whose vertical neighbour is missing is, however, assigned the boundary value even though it lies slightly inside the boundary in z, which is the main source of the discretisation error discussed in Section 3.1.

2.4 Sparse linear algebra

Collecting the active nodes in a single index k gives a linear system $A\boldsymbol{\psi} = \boldsymbol{b}$, where $A \in \mathbb{R}^{N\times N}$ has at most five non-zeros per row. It is assembled in LIL format (cheap incremental insertion), converted to CSR, and solved with `scipy.sparse.linalg.spsolve` (sparse LU, SuperLU). Dirichlet rows read $\psi_k = 1$ with $b_k = 1$; interior rows read $(A\boldsymbol{\psi})_k = S_k$. For Solov'ev, a single solve ($\approx 0.1\,\mathrm{s}$ for $N \approx 6000$) is enough. The matrix does *not* depend on $\boldsymbol{\psi}$ in the nonlinear case, only the right-hand side does; the present code nevertheless refactorises at each Picard step, so an easy speed-up is to compute one `splu` factorisation and reuse it.

2.5 Picard iteration for the nonlinear case

For the nonlinear source, one step of the iteration is a map $P : \boldsymbol{\psi} \mapsto \boldsymbol{\psi}^+$ defined as

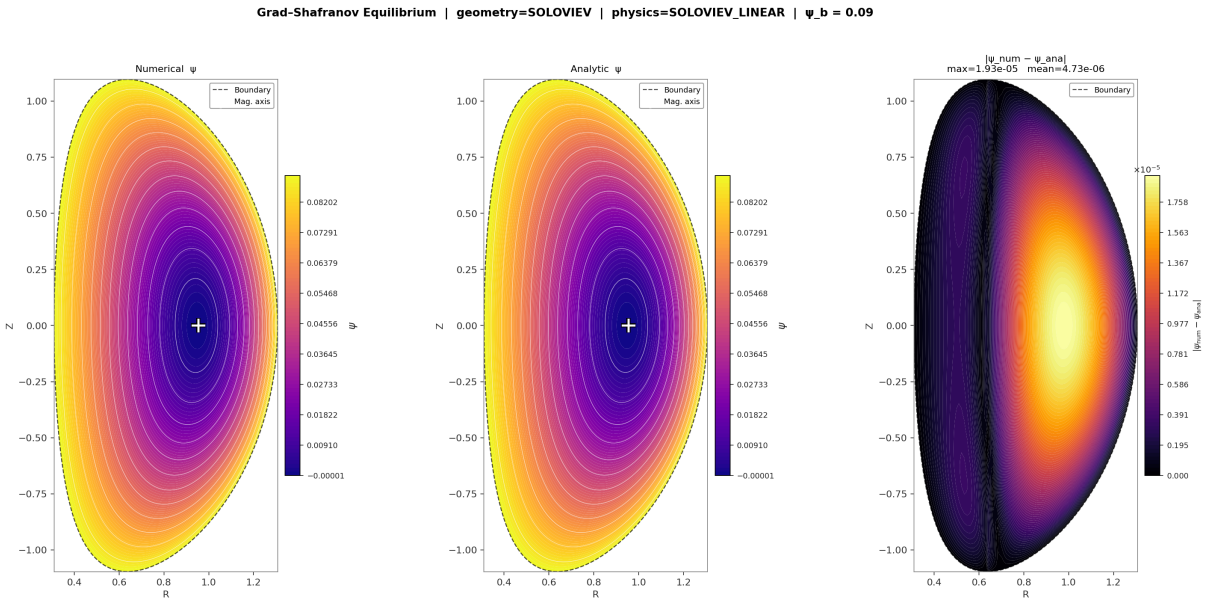


Figure 2: Linear Solov'ev benchmark, $\Delta s = 0.02$. Left: numerical ψ; middle: analytic ψ; right: absolute error.

1. compute the normalised flux $\psi_n = (\psi - \psi_{\min})/(\bar{\psi}_b - \psi_{\min})$ at every node, clipped to $[0, 1]$ ($\psi_{\min}$ is the minimum over the grid, $\bar{\psi}_b$ the mean boundary value);
2. evaluate the right-hand side $b_k = S(r_k, \psi_n^{(k)})$ at interior nodes ($b_k = 1$ on the boundary);
3. solve $A\boldsymbol{u} = \boldsymbol{b}$;
4. renormalise $\boldsymbol{\psi}^+ = (\boldsymbol{u} - u_{\min})/(\bar{u}_b - u_{\min})$.

The iteration (a plain fixed-point or Picard iteration on the nonlinear source, see e.g. Jardin [2]) is started from $\psi = 0$ in the interior and $\psi = 1$ on the boundary, $\boldsymbol{\psi}^{k+1} = P(\boldsymbol{\psi}^k)$. A solution of the normalised GS problem is a fixed point $P(\boldsymbol{\psi}) = \boldsymbol{\psi}$, so the stopping test is the *fixed-point residual* $\|P(\boldsymbol{\psi}) - \boldsymbol{\psi}\|/\|\boldsymbol{\psi}\| < 10^{-9}$; for every nonlinear run the code reports it, together with $\|P(P(\boldsymbol{\psi})) - \boldsymbol{\psi}\|/\|\boldsymbol{\psi}\|$, and prints a warning if $\boldsymbol{\psi}$ is not a fixed point (and if the iteration looks like a period-two cycle). Two structural remarks:

- Because A is linear and step 4 removes any overall factor, *only the ratio p_0/α matters* for the normalised map: scaling $(p_0, \alpha) \to s\,(p_0, \alpha)$ scales $\boldsymbol{u} - \mathbf{1}$ by s, which the renormalisation undoes.
- López *et al.* instead hold the total plasma current and the on-axis pressure fixed by adjusting a coefficient in the f^2 profile at every iteration (their Eqs. (15)–(17)); the code here does not implement that constraint.

The geometry is validated at start-up: the Shafranov boundary requires $a < R_0$, since otherwise the plasma crosses $R = 0$ where the $1/R$ term of Δ^* is singular.

3 Results

3.1 Benchmark against the Solov'ev solution

We use the parameters of [1]: $B_0 = 0.5\,\mathrm{T}$, $R_0 = 0.95$, $\kappa_0 = 2.2$, $q_0 = 1.1$, $\psi_b = 0.09$, on the Solov'ev contour, in mode `SOLOVIEV_LINEAR`. Figure 2 shows the numerical and exact fluxes and their difference for $\Delta s = 0.02$ ($N = 5041$ unknowns, a 141×71 tensor grid of which every active node is used): the maximum error is 1.9×10^{-5} and the mean error 4.7×10^{-6}, i.e. 2×10^{-4} relative to the peak value of ψ. The error is largest around the magnetic axis and decays towards the boundary, where ψ is prescribed.

Grid convergence. Table 1 and Figure 3 show the error as the boundary step Δs is refined. A least-squares fit on the log–log data gives an observed order of 1.88 (max norm) and 1.79 (mean), i.e. close to the second order expected for the stencils. The slightly sub-quadratic max-norm slope is consistent with the first-order treatment of the vertical boundary condition noted above. For reference, López *et al.* report on a 241×241 mesh an error along the r axis that does not exceed 10^{-5} (their Fig. 4c). Our run at $\Delta s = 0.012$ (233×233 tensor grid, $N = 13689$ active nodes) has a maximum error of 7.0×10^{-6} over the whole grid, which is of the same order and slightly below that bound at comparable resolution; we have not reproduced their SOR iteration and the comparison is only as precise as their one-line statement.

Table 1: Solov'ev grid-convergence study (error in ψ in the units of $\psi_b = 0.09$).

Δs	N	max error	mean error
0.200	81	1.34×10^{-3}	2.65×10^{-4}
0.140	144	7.34×10^{-4}	1.45×10^{-4}
0.100	256	4.08×10^{-4}	8.40×10^{-5}
0.070	441	2.14×10^{-4}	4.94×10^{-5}
0.050	841	1.11×10^{-4}	2.61×10^{-5}
0.035	1681	5.59×10^{-5}	1.33×10^{-5}
0.025	3249	2.94×10^{-5}	7.14×10^{-6}
0.018	6241	1.53×10^{-5}	3.72×10^{-6}
0.012	13689	6.99×10^{-6}	1.74×10^{-6}

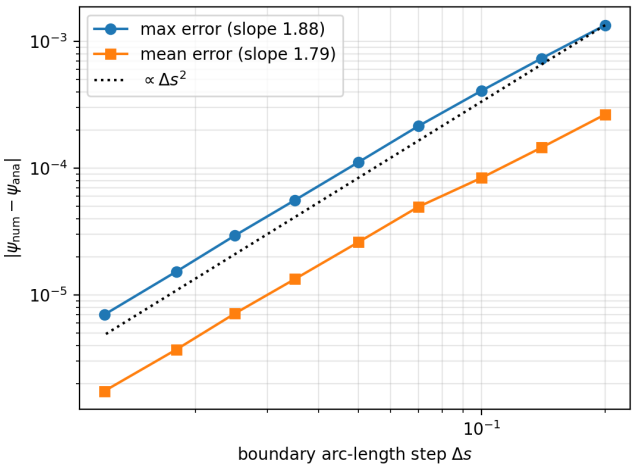


Figure 3: Error versus boundary step; dotted: $\propto \Delta s^2$.

3.2 Validation of the nonlinear code path

There is no analytic solution for a generic nonlinear source, so the nonlinear path is validated in two ways (`scripts/nonlinear_validation.py`).

Solov'ev through the Picard path. Running the Picard machinery (`CUSTOM` source equal to the exact Solov'ev source, plain iteration) must reproduce the analytic solution. It does, converging in 2 iterations (one solve plus one confirming step) to a residual below 10^{-12}, with maximum errors from 5×10^{-4} at $N = 144$ down to 1.7×10^{-5} at $N = 3249$, an observed order of 1.9. The errors are not strictly monotone because the renormalisation uses the discrete minimum of ψ, which exceeds the true axis value by an amount that varies with the (non-nested) mesh.

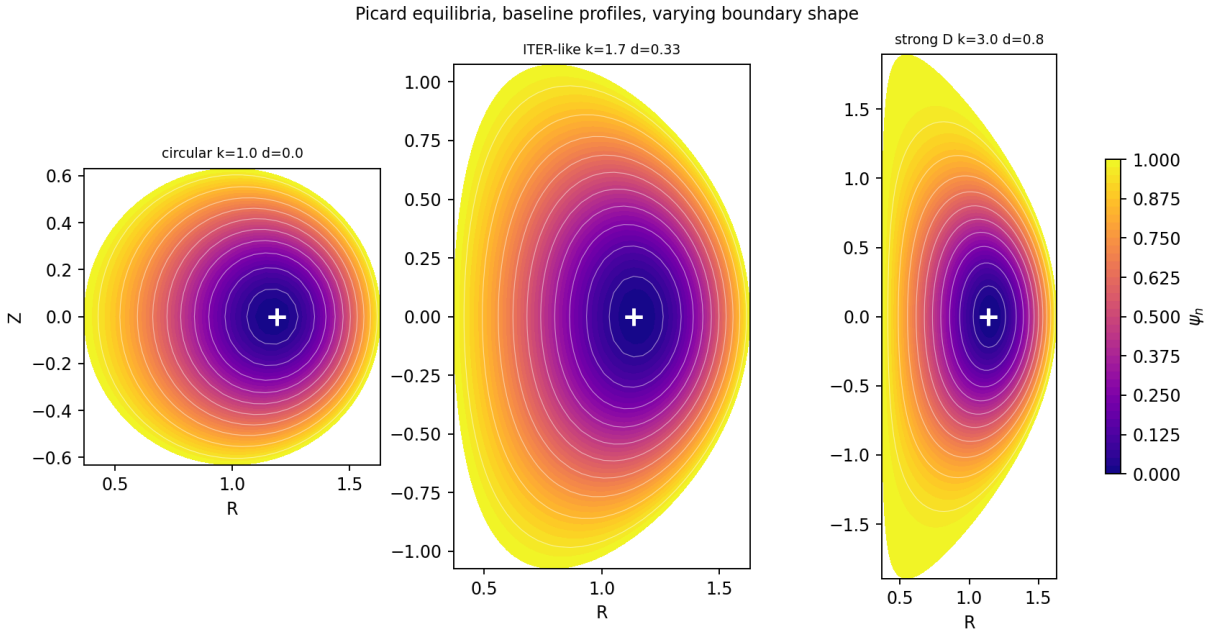


Figure 4: Flux surfaces (ψ_n contours; white +: magnetic axis) at the baseline profiles for three boundary shapes: circular, ITER-like, strong D.

Grid convergence of a nonlinear case. For the ITER-like boundary with baseline profiles we track the amplitude functional $C = 1 - u_{\min}$ (the flux difference between boundary and axis in the unnormalised solve at the fixed point) for $N = 144$ up to 6889. It converges, from 0.16780 to 0.16729, with a fit $C(\Delta s) = C_\infty + c\,\Delta s^p$ giving $C_\infty = 0.16723$ and $p \approx 1.2$; the plain iteration needs 27–28 iterations at every resolution and the residual is $< 4 \times 10^{-13}$ throughout. The observed order is closer to one than to two, consistent with the first-order treatment of the vertical boundary condition and with non-nested meshes. We therefore claim convergence, but not second-order accuracy, for the nonlinear solutions.

3.3 Nonlinear equilibria: profiles and shapes

We use the Shafranov boundary with $R_0 = 0.95$, $a = 0.6$ ($a/R_0 = 0.63$), $\Delta s = 0.05$, and plain Picard iteration. Baseline exponents are $(s_{p1}, s_{p2}) = (2, 2)$, $(s_{f1}, s_{f2}) = (2, 1)$ and $p_0 = \alpha = 1$. We first vary the boundary shape, then the profiles at the ITER-like shape $\kappa = 1.7$, $\delta = 0.33$. Table 2 lists the iterations needed, the fixed-point residual, the position of the magnetic axis (parabolic interpolation through the three midplane nodes around the minimum) and its displacement from the geometric centre of the midplane chord, normalised by the half-width of that chord.

Table 2: Nonlinear cases (plain Picard). All reach a true fixed point of P.

case	(κ, δ)	N	iter.	$\|P\boldsymbol{\psi} - \boldsymbol{\psi}\|/\|\boldsymbol{\psi}\|$	R_{axis}	shift/half-width
circular	(1.0, 0.00)	441	22	1.9×10^{-11}	1.174	+0.275
ITER-like (baseline)	(1.7, 0.33)	961	23	1.6×10^{-11}	1.152	+0.240
strong D	(3.0, 0.80)	2209	30	2.6×10^{-11}	1.151	+0.238
peaked p', $(s_{p1}, s_{p2}) = (2, 4)$	(1.7, 0.33)	961	26	2.3×10^{-11}	1.149	+0.236
broad p', FF', (4, 1) and (4, 1)	(1.7, 0.33)	961	15	1.3×10^{-11}	1.147	+0.232
current-driven, p_0=0.2, α=2	(1.7, 0.33)	961	20	1.4×10^{-11}	1.078	+0.124
pressure-driven, p_0=3, α=0.3	(1.7, 0.33)	961	25	3.7×10^{-11}	1.189	+0.300

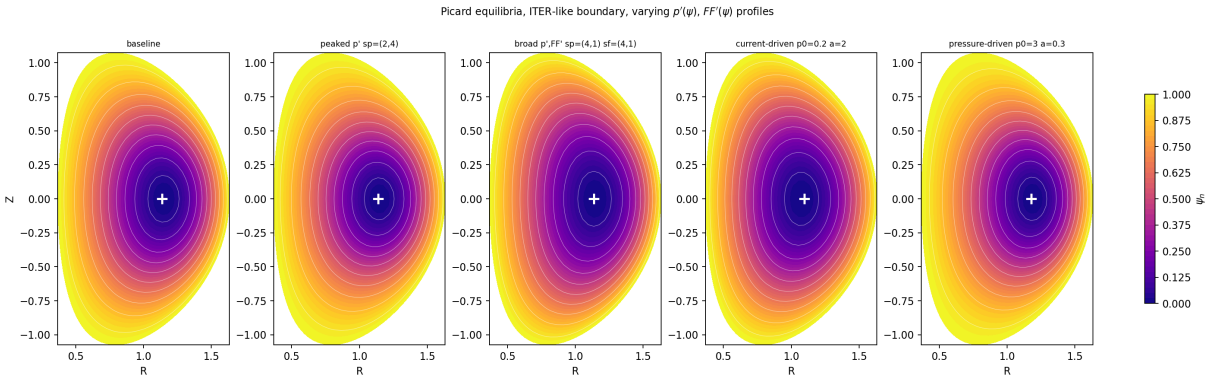


Figure 5: Flux surfaces at the ITER-like boundary for five choices of the profiles.

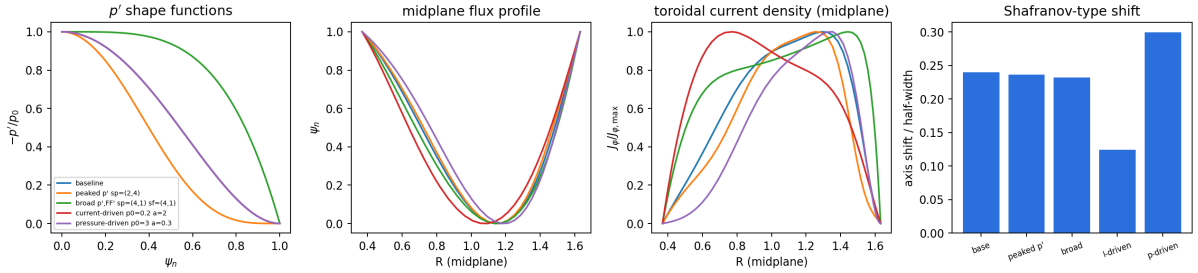


Figure 6: Left to right: shape of $-p'$; midplane flux profile; midplane toroidal current density $J_\varphi \propto S/R$ (each case normalised to its maximum); normalised axis shift.

What the cases show.

- *Toroidicity shifts the axis outward even for a circular boundary.* The circular case is not symmetric under $R \to 2R_0 - R$ (the operator contains $1/R$), and the axis sits at $+0.28$ half-widths outward of the geometric centre. This is the Shafranov shift: the flux surfaces are compressed on the outboard side and the axis moves outward relative to the centre of the boundary. Increasing the shaping to $\kappa = 1.7, \delta = 0.33$ or to $\kappa = 3, \delta = 0.8$ changes the normalised shift only slightly ($0.275 \to 0.240 \to 0.238$), while the surfaces follow the D-shape (Fig. 4).
- *The shift is controlled by the pressure-to-current ratio, not by the peaking of p'.* At fixed ratio $p_0/\alpha = 1$, changing the exponents (peaked, baseline or broad p' and FF') moves the shift only between 0.232 and 0.240. In contrast, the ratio does the work: $p_0/\alpha = 0.1$ gives 0.124 and $p_0/\alpha = 10$ gives 0.300, a factor 2.4. This is consistent with the dependence of the Shafranov shift on poloidal beta: more pressure support relative to the poloidal-field-producing current pushes the axis outward.
- *The current density profile follows suit.* Because $J_\varphi \propto S/R$ and the pressure term of S carries a factor R^2, the midplane current density peaks outboard of the axis for pressure-driven profiles and inboard for current-driven ones (Fig. 6, third panel), but it never vanishes on the axis.
- *Cost.* All cases converge to residuals of 10^{-11} in 15–30 plain Picard iterations; broader profiles converge fastest (spectral radius 0.22 versus 0.44 for the peaked p'; Table 3).

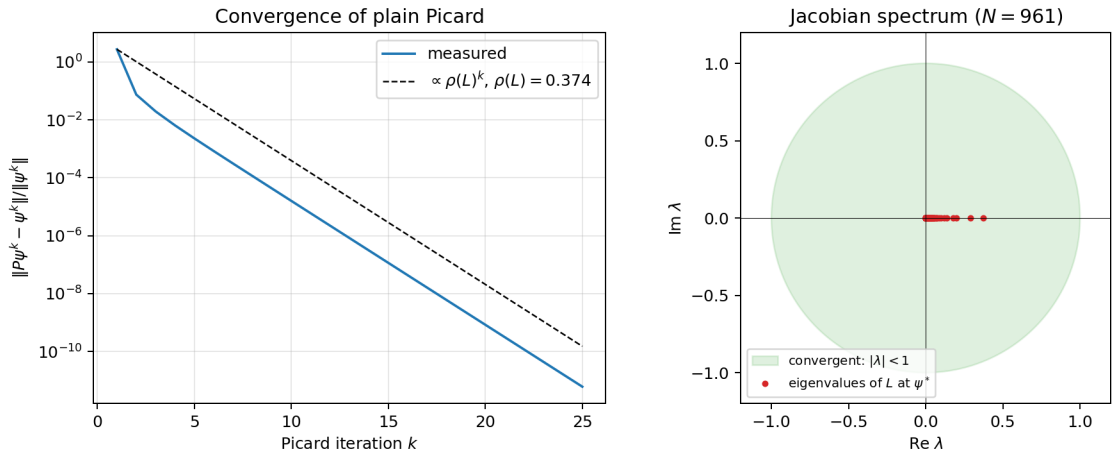


Figure 7: Left: fixed-point residual of plain Picard (ITER-like boundary, baseline profiles, $N = 961$) and the decay $\rho(L)^k$ predicted by the spectral radius. Right: eigenvalues of the Picard Jacobian L at the fixed point against the unit disc.

4 Convergence of the Picard iteration

4.1 Linear stability

Let P be the one-step Picard map of Section 2.5, $\boldsymbol{\psi}^*$ a fixed point and $L = \partial P/\partial\boldsymbol{\psi}|_{\boldsymbol{\psi}^*}$ its Jacobian. Near $\boldsymbol{\psi}^*$ the error evolves as $\boldsymbol{e}^{k+1} \approx L\boldsymbol{e}^k$, so plain Picard converges if and only if the spectral radius $\rho(L) < 1$, with asymptotic rate $\rho(L)$ per iteration; reducing the residual by a factor ε takes about $\ln\varepsilon/\ln\rho(L)$ iterations. We build L column by column by central differences, $L\boldsymbol{e}_j \approx [P(\boldsymbol{\psi}^* + \eta\boldsymbol{e}_j) - P(\boldsymbol{\psi}^* - \eta\boldsymbol{e}_j)]/2\eta$ with $\eta = 10^{-6}\|\boldsymbol{\psi}^*\|$, and diagonalise it densely (grids up to $N \approx 10^3$). For larger grids the code can instead estimate the leading eigenvalues matrix-free with `scipy.sparse.linalg.eigs` (`SPECTRAL_ANALYSIS`).

4.2 Measurements

All numbers below come from `scripts/spectral_study.py` (output in `results/spectral_study.json`).

Main case ($N = 961$). Plain Picard reaches a residual of 10^{-11} in 25 iterations. The spectrum of L at the fixed point is real and non-negative, $\lambda \in [0, 0.374]$ (largest imaginary part 2×10^{-6}), so $\rho(L) = 0.374$ and the predicted number of iterations is $\ln 10^{-11}/\ln 0.374 = 25.7$, matching the observed 25 (Fig. 7).

Table 3: Plain-Picard iterations (to a residual of 10^{-11}) and spectral radius of L at the fixed point, dense Jacobian. In every case the spectrum is real and non-negative to within 4×10^{-4} in imaginary part.

case	N	iterations	$\rho(L)$
ITER-like, baseline	484	25	0.380
circular	225	24	0.374
strong D $(3.0, 0.8)$	576	33	0.484
pressure-driven, $p_0/\alpha = 10$	484	28	0.420
current-driven, $p_0/\alpha = 0.1$	484	22	0.316
broad profiles	484	17	0.219
peaked p'	484	29	0.437

Reading. With peaked-current profiles the Jacobian has a real, non-negative spectrum inside $[0, 0.5)$, so plain Picard converges monotonically with rate 0.22–0.48 in all seven configurations tested, and $\rho(L)$ predicts the iteration count well. Broader profiles converge fastest; strong shaping and peaked p' slowest. We have not tested hollow-current, high-beta or very strongly shaped profiles, where the spectrum could look different.

4.3 Profile parametrisation: why the shapes are on p' and FF'

`scripts/profile_parametrisation_check.py` runs plain Picard on the ITER-like boundary for sources built in different ways (Table 4). When the shapes $(1-\psi_n^2)^2$ are imposed on p and F^2 and then differentiated, the derivatives are proportional to ψ_n and vanish on the axis: $S(\psi_n = 0) = 0$ and $J_\varphi = 0$ on the magnetic axis (a hollow current). In that case plain Picard does not converge: after 800 iterations $\|P\boldsymbol{\psi} - \boldsymbol{\psi}\|/\|\boldsymbol{\psi}\| \approx 0.11$–$0.12$ while $\|P(P\boldsymbol{\psi}) - \boldsymbol{\psi}\| \sim 10^{-12}$, i.e. the iterates oscillate between two states, a period-two cycle of P. This residual is the same at $\Delta s = 0.10, 0.05$ and 0.035, so it is not a resolution effect. With exponents $(1, 1, 1)$ (source independent of ψ) or $(1, 2, 1)$ (nonzero p' on the axis, source depends on ψ) the same code converges to residuals of 10^{-11}–10^{-13}, as does the default derivative-shaped source.

Table 4: Plain Picard (up to 800 iterations) for different ways of imposing the profile shapes ($\Delta s = 0.05$; the other resolutions are in `results/profile_parametrisation_check.json`).

shape imposed on	iterations	$\|P\boldsymbol{\psi} - \boldsymbol{\psi}\|/\|\boldsymbol{\psi}\|$	$\|P^2\boldsymbol{\psi} - \boldsymbol{\psi}\|/\|\boldsymbol{\psi}\|$
p', FF' (default)	23	1.6×10^{-11}	2.2×10^{-11}
p, F^2, exponents $(2, 2, 2)$	800	1.1×10^{-1}	3.8×10^{-13}
p, F^2, exponents $(1, 1, 1)$	2	5×10^{-14}	7×10^{-14}
p, F^2, exponents $(1, 2, 1)$	15	1.1×10^{-11}	1.4×10^{-11}

We have not proved *why* a vanishing source on the axis produces the oscillation; the evidence is this controlled comparison. The practical rule is to impose the profile shapes on p' and FF' for peaked-current equilibria, and to use the fixed-point residual, not the size of the last step, to decide whether a nonlinear run has produced a solution.

4.4 Outlook

- Implement the constraint of López *et al.* (fixed total current and axis pressure via a coefficient adjusted each iteration), and compare.
- Reuse one sparse LU factorisation across Picard iterations, since A does not change.
- A free-boundary variant on top of the same matrix machinery; inverse reconstruction of p', FF' from synthetic data.
- Quantitative checks of the nonlinear solutions against a published nonlinear equilibrium, which this work does not provide.

A Derivation of the Grad–Shafranov equation

This derivation follows the approach of Ben Dudson's lecture series "Plasma fluid theory" [3]. The three ingredients are $\boldsymbol{B} = \nabla \times \boldsymbol{A}$, Ampère's law $\mu_0 \boldsymbol{J} = \nabla \times \boldsymbol{B}$ (static equilibrium, so the displacement current is irrelevant) and force balance $\boldsymbol{J} \times \boldsymbol{B} = \nabla p$. The key assumption is axisymmetry, $\partial_\varphi \equiv 0$ on every *scalar* field. The cylindrical basis vectors still depend on φ,

$$\partial_\varphi \hat{\boldsymbol{r}} = \hat{\boldsymbol{\varphi}}, \qquad \partial_\varphi \hat{\boldsymbol{\varphi}} = -\hat{\boldsymbol{r}}, \tag{11}$$

a subtle point that is precisely what produces the elliptic operator Δ^*. We work in statics, $\partial_t \equiv 0$.

A.1 Step I: vector potential and flux function

With $\partial_\varphi = 0$, $\boldsymbol{B} = \nabla \times \boldsymbol{A}$ reads

$$\boldsymbol{B} = -\frac{1}{r}\frac{\partial}{\partial z}(rA_\varphi)\,\hat{\boldsymbol{r}} + \frac{1}{r}\frac{\partial}{\partial r}(rA_\varphi)\,\hat{\boldsymbol{z}} + B_\varphi\hat{\boldsymbol{\varphi}}, \qquad B_\varphi = \partial_z A_r - \partial_r A_z. \tag{12}$$

Define the poloidal flux function $\psi := rA_\varphi$. Then

$$\boldsymbol{B} = -\frac{1}{r}\frac{\partial\psi}{\partial z}\hat{\boldsymbol{r}} + \frac{1}{r}\frac{\partial\psi}{\partial r}\hat{\boldsymbol{z}} + B_\varphi\hat{\boldsymbol{\varphi}} = \frac{1}{r}\nabla\psi \times \hat{\boldsymbol{\varphi}} + B_\varphi\hat{\boldsymbol{\varphi}}. \tag{13}$$

Field lines lie on flux surfaces. The poloidal part of $\boldsymbol{B}$ is perpendicular to $\nabla\psi$. A vector $\boldsymbol{u}$ tangent to a surface $\psi = \psi_0$ satisfies $\partial_{\boldsymbol{u}}\psi = \boldsymbol{u}\cdot\nabla\psi = 0$, so $\nabla\psi$ is normal to the surface; since the poloidal field is also perpendicular to $\nabla\psi$, it is parallel to the surface. Hence field lines lie on the surfaces $\psi = \text{const}$.

A.2 Step II: Ampère's law

Compute $\nabla \times \boldsymbol{B}$ term by term. Using $\nabla\times(\boldsymbol{X}\times\boldsymbol{Y}) = (\nabla\cdot\boldsymbol{Y})\boldsymbol{X} + (\boldsymbol{Y}\cdot\nabla)\boldsymbol{X} - (\nabla\cdot\boldsymbol{X})\boldsymbol{Y} - (\boldsymbol{X}\cdot\nabla)\boldsymbol{Y}$ with $\boldsymbol{X} = \frac{1}{r}\nabla\psi$, $\boldsymbol{Y} = \hat{\boldsymbol{\varphi}}$:

- $\nabla\cdot\hat{\boldsymbol{\varphi}} = 0$;
- $(\boldsymbol{X}\cdot\nabla)\hat{\boldsymbol{\varphi}} = 0$, since $\boldsymbol{X}$ has no φ-component and $\hat{\boldsymbol{\varphi}}$ depends only on φ;
- $(\hat{\boldsymbol{\varphi}}\cdot\nabla)\boldsymbol{X} = \frac{1}{r}\partial_\varphi\Big(\frac{1}{r}\frac{\partial\psi}{\partial r}\hat{\boldsymbol{r}} + \frac{1}{r}\frac{\partial\psi}{\partial z}\hat{\boldsymbol{z}}\Big) = \frac{1}{r^2}\frac{\partial\psi}{\partial r}\hat{\boldsymbol{\varphi}}$, using $\partial_\varphi\hat{\boldsymbol{r}} = \hat{\boldsymbol{\varphi}}$ and $\partial_\varphi\hat{\boldsymbol{z}} = 0$;
- $\nabla\cdot\boldsymbol{X} = \nabla\cdot(\frac{1}{r}\nabla\psi) = \nabla(\frac{1}{r})\cdot\nabla\psi + \frac{1}{r}\nabla^2\psi = -\frac{1}{r^2}\frac{\partial\psi}{\partial r} + \frac{1}{r}\nabla^2\psi$, where $\nabla^2\psi = \partial_r^2\psi + \frac{1}{r}\partial_r\psi + \partial_z^2\psi$ is the axisymmetric scalar Laplacian.

Therefore

$$\begin{aligned}\nabla\times\Big(\frac{1}{r}\nabla\psi\times\hat{\boldsymbol{\varphi}}\Big) &= \Big[\frac{1}{r^2}\frac{\partial\psi}{\partial r} + \frac{1}{r^2}\frac{\partial\psi}{\partial r} - \frac{1}{r}\Big(\partial_r^2\psi + \frac{1}{r}\partial_r\psi + \partial_z^2\psi\Big)\Big]\hat{\boldsymbol{\varphi}} \\ &= -\frac{1}{r}\Big[\partial_r^2\psi - \frac{1}{r}\partial_r\psi + \partial_z^2\psi\Big]\hat{\boldsymbol{\varphi}} = -\frac{1}{r}\Delta^*\psi\,\hat{\boldsymbol{\varphi}}.\end{aligned} \tag{14}$$

For the toroidal part, write $B_\varphi\hat{\boldsymbol{\varphi}} = (rB_\varphi)\,\hat{\boldsymbol{\varphi}}/r$ and use $\nabla\times(\hat{\boldsymbol{\varphi}}/r) = 0$ (curl in cylindrical coordinates):

$$\nabla\times(B_\varphi\hat{\boldsymbol{\varphi}}) = \nabla(rB_\varphi)\times\frac{\hat{\boldsymbol{\varphi}}}{r}. \tag{15}$$

Defining $f := rB_\varphi$, Ampère's law becomes

$$\mu_0\boldsymbol{J} = \underbrace{\frac{1}{r}\nabla f\times\hat{\boldsymbol{\varphi}}}_{\text{poloidal current}} \underbrace{-\frac{1}{r}\Delta^*\psi\,\hat{\boldsymbol{\varphi}}}_{\mu_0 J_\varphi\,\hat{\boldsymbol{\varphi}}}. \tag{16}$$

A.3 Step III: force balance

Axisymmetry implies $\nabla p\cdot\hat{\boldsymbol{\varphi}} = 0$ (and $\nabla f\cdot\hat{\boldsymbol{\varphi}} = 0$).

$p = p(\psi)$. From $\boldsymbol{J}\times\boldsymbol{B} = \nabla p$ we get $\boldsymbol{B}\cdot\nabla p = 0$. Inserting $\boldsymbol{B}$ and using that only the poloidal part contributes,

$$0 = \nabla p\cdot\Big(\frac{1}{r}\nabla\psi\times\hat{\boldsymbol{\varphi}}\Big) \implies (\nabla\psi\times\nabla p)\cdot\hat{\boldsymbol{\varphi}} = 0 \implies \partial_r\psi\,\partial_z p - \partial_z\psi\,\partial_r p = 0. \tag{17}$$

So $\nabla p \parallel \nabla\psi$: p and ψ have the same level sets, hence $p = p(\psi)$ and $\nabla p = \frac{\mathrm{d}p}{\mathrm{d}\psi}\nabla\psi$.

$f = f(\psi)$**.** Similarly $\boldsymbol{J} \cdot \nabla p = 0$. The $J_\varphi \hat{\boldsymbol{\varphi}}$ term drops out because $\nabla p \cdot \hat{\boldsymbol{\varphi}} = 0$, leaving $\nabla p \cdot (\nabla f \times \hat{\boldsymbol{\varphi}}) = 0$, i.e. $\hat{\boldsymbol{\varphi}} \cdot (\nabla p \times \nabla f) = 0$. Both ∇p and ∇f lie in the rz-plane, so their cross product is purely toroidal, and its vanishing means $\nabla f \parallel \nabla p \parallel \nabla\psi$. Hence $f = f(\psi)$ and $\nabla f = \frac{\mathrm{d}f}{\mathrm{d}\psi}\nabla\psi$.

Force balance. With $\boldsymbol{J} = J_\varphi \hat{\boldsymbol{\varphi}} + \frac{1}{\mu_0 r}\nabla f \times \hat{\boldsymbol{\varphi}}$ and $\boldsymbol{B} = \frac{1}{r}\nabla\psi \times \hat{\boldsymbol{\varphi}} + \frac{f}{r}\hat{\boldsymbol{\varphi}}$, the product of the two poloidal parts vanishes (they are parallel), leaving

$$\begin{aligned} \boldsymbol{J} \times \boldsymbol{B} &= \frac{J_\varphi}{r}\,\hat{\boldsymbol{\varphi}} \times (\nabla\psi \times \hat{\boldsymbol{\varphi}}) - \frac{f}{\mu_0 r^2}\,\hat{\boldsymbol{\varphi}} \times (\nabla f \times \hat{\boldsymbol{\varphi}}) \\ &= \frac{J_\varphi}{r}\nabla\psi - \frac{f}{\mu_0 r^2}\nabla f = \Big(\frac{J_\varphi}{r} - \frac{f}{\mu_0 r^2}\frac{\mathrm{d}f}{\mathrm{d}\psi}\Big)\nabla\psi, \end{aligned} \tag{18}$$

where $\hat{\boldsymbol{\varphi}} \times (\boldsymbol{a} \times \hat{\boldsymbol{\varphi}}) = \boldsymbol{a}$ for $\boldsymbol{a} \perp \hat{\boldsymbol{\varphi}}$ was used. Equating with $\nabla p = \frac{\mathrm{d}p}{\mathrm{d}\psi}\nabla\psi$:

$$\frac{J_\varphi}{r} - \frac{f}{\mu_0 r^2}\frac{\mathrm{d}f}{\mathrm{d}\psi} = \frac{\mathrm{d}p}{\mathrm{d}\psi} \implies -\mu_0 r J_\varphi = -\mu_0 r^2 \frac{\mathrm{d}p}{\mathrm{d}\psi} - f\frac{\mathrm{d}f}{\mathrm{d}\psi}. \tag{19}$$

Finally, Step II gave $\mu_0 J_\varphi = -\frac{1}{r}\Delta^*\psi$, i.e. $-\mu_0 r J_\varphi = \Delta^*\psi$, and so we obtain the Grad–Shafranov equation

$$\boxed{\Delta^*\psi = -\mu_0 r^2 \frac{\mathrm{d}p}{\mathrm{d}\psi} - f\frac{\mathrm{d}f}{\mathrm{d}\psi}}. \tag{20}$$

A.4 Non-dimensionalisation

In many applications one non-dimensionalises so as to absorb μ_0, after which the equation reads $\tilde{\Delta}^*\tilde{\psi} = -\tilde{r}\tilde{J}_\varphi$ with $\tilde{J}_\varphi = \tilde{r}\,\mathrm{d}\tilde{p}/\mathrm{d}\tilde{\psi} + (\tilde{f}/\tilde{r})\,\mathrm{d}\tilde{f}/\mathrm{d}\tilde{\psi}$: a slightly modified Poisson equation with a nonlinear source on the right-hand side.

References

[1] J. E. López, E. A. Orozco and V. D. Dougar-Zhabon, *Fixed boundary Grad–Shafranov solver using finite difference method in nonhomogeneous meshgrid*, J. Phys.: Conf. Ser. **1159**, 012017 (2019). doi:10.1088/1742-6596/1159/1/012017

[2] S. C. Jardin, *Computational Methods in Plasma Physics*, CRC Press (2010).

[3] B. Dudson, *Plasma fluid theory*, lecture series (YouTube).